Memory Efficient Doubly Linked List – Insert After Position –Space Complexity and Algorithm

Program :

```
void insertAfterElement(int data, int element)
{
    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Search for element
    while (curr != nullptr && curr->data != element)
    {
        next = XOR(prev, curr->npx);
        prev = curr;
        curr = next;
    }

    if (curr == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    // Element found at 'curr'. Insert 'newNode' between
'curr' and 'next'.
    next = XOR(prev, curr->npx); // Decode the node after
curr

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    // Update pointers using XOR logic
    newNode->npx = XOR(curr, next);
    curr->npx = XOR(prev, newNode);
```

```
    if (next != nullptr)
    {
        Node *nextNext = XOR(curr, next->npx);
        next->npx = XOR(newNode, nextNext);
    }

    cout << data << " inserted after " << element << "." <<
endl;
}
…..
       case 4:
            cout << "Enter data to insert: ";
            cin >> data;
            cout << "Enter the element to insert after: ";
            cin >> element;
            insertAfterElement(data, element);
            break;
```

Space Complexity Analysis

XOR Linked List (Insert After Element)

1. *Input Space Complexity*

- ***Parameter – only Definition***:
    - ***Function Parameters***: *int data (the value to insert)and int element (the target value to search for).*
    - ***Analysis***: *Both are primitive integers.They take up a constant amount of space regardless of the list size.*
    - ***Complexity***: $O(1)$

- ***Full Data Definition***:
    - ***Function Data***: *The parameters + the existing XOR linked list structure.*
    - ***Analysis***: *The function must search through the existing list of n nodes.The* input*context for this operation is the entire data structure.*
    - ***Complexity***: $O(n)$

---

2. ***Auxiliary Space Complexity***

Definition: *This measures the* extra*or temporary memory the function uses to perform its task.*

- ***Node Allocation***: $Node * newNode = (Node *)malloc(sizeof(Node))$;. *This is the most significant part. We allocate memory for exactly* ***one*** *node.*
- ***Local Variables***: *The function uses three pointers* $(curr, prev, next)$ *and one temporary pointer* $nextNext$.
    - ***Analysis***: *These are stack − allocated variables.Their space requirement is fixed* $(usually\ 8\ bytes\ per\ pointer\ on\ a\ 64-bit\ system)$*and does not increase even if the list has a billion nodes.*
- ***XOR Operations***: *The bitwise math is performed in − place using CPU registers.*
- ***Traversal***: *Since the search is done via an iterative* $while$ *loop, there is no recursion depth to worry about.*

Conclusion: *The amount of extra memory used is fixed and does not grow as the size of the linked list* (n) *grows.*

- ***Auxiliary Space Complexity***: $O(1)$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\underbrace{Space = O(1)}_{Auxiliary} + \underbrace{O(1)}_{Input} = O(1).$$

- **Holistic Interpretation (Full-INFO input):**

$$\underbrace{Space = O(1)}_{Auxiliary} + \underbrace{O(n)}_{Input} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

***Algorithm*: *Insert After Element* (*XOR Doubly Linked List*)**

***XORINSAFTER*(*START*, *ITEM*, *ELEMENT*):**

$$\left(\left(\begin{array}{c}(Node*)malloc\big(sizeof(Node)\big)represents\\ the\ availability\ of\ space\ for\ the\ data;\ let's\ name\ it\ AVAIL.\end{array}\right)\right)$$

1. [***INITIALIZE SEARCH***]
   Set *CURRPTR* := ***START***
   Set *PREVPTR* := *NULL*

2. [***SEARCH FOR ELEMENT ELEMENT***]
   Repeat while $CURRPTRPTR \neq NULL$ *AND* $INFO[CURRPTR] \neq ELEMENT$:
   $a.$ *Set* $NEXTTEMPPTR := PREVPTRPTR \oplus NEXTXORPTR[CURRPTR]$
   $b.$ *Set* $PREVPTR := CURRPTR$
   $c.$ *Set* $CURRPTR := NEXTTEMPPTR$
   [*End of Loop*]

3. [***ELEMENT FOUND***?]
   If $CURRPTR = NULL$, *then*:
   Write: "*Element* ", *ELEMENT*, " *not found in the list.*"
   Return and ***EXIT***.
   [*End of If structure*]

4. [***PREPARE NEW NODE***]
   $a.$ *Set* $NEXTTEMPPTR$:
   $$= PREVPTR \oplus NEXTXORPTR[CURRPTR]\left(\begin{array}{c}Decode\ the\ node\\ currently\ following\\ CURR\end{array}\right)$$
   $b.$ *Set* $NEWPTR := AVAIL$
   $c.$ *If* $NEWPTR = NULL$, *then*: *Write*: "*Error*: *Memory allocation failed.*"
   Return and ***EXIT***.
   [*End of If structure*]
   $d.$ *Set* $INFO[NEWPTR] := ITEM$

5. [***LINK NEW NODE***]
   $a.$ *Set* $NEXTXORPTR[NEWPTR]$:
   $$= CURRPTR \oplus NEXTTEMPPTR\left(\begin{array}{c}The\ new\ node's\ neighbours\\ are\ the\ target\ node\\ and\ the\ original\ successor\end{array}\right)$$

$b.Set\ NEXTXORPTR[CURRPTR]$:

$$= PREVPTR \oplus NEWPTR \left(\begin{array}{c} Update\ target\ node's\ NPX: \\ its\ previous\ stays\ the\ same, \\ but\ its\ NEXT \\ is\ now\ the\ new\ node \end{array} \right)$$

6. [***UPDATE SUCCESSOR NODE IF IT EXISTS***]
   $If\ NEXTTEMPPTR \neq NULL, then$:
   $a.Set$ ***NEXTNEXTTEMPPTR*** :

$$= CURRPTR \oplus NEXTXORPTR[NEXTTEMPPTR] \left(\begin{array}{c} Decode\ the\ node \\ following \\ NEXT \end{array} \right)$$

   $b.Set\ NEXTXORPTR[NEXTTEMPPTR]$:

$$= NEWPTR \oplus \textbf{\textit{NEXTNEXTTEMPPTR}} \left(\begin{array}{c} Update\ successor's\ NPX: \\ its\ previous\ is\ now\ the \\ new\ node\ instead\ of\ CURR \end{array} \right)$$

   [$End\ of\ If\ structure$]

7. [***PRINT MESSAGE***]
   $Write$: $ITEM$, " $inserted\ after$ ", $ELEMENT$

8. ***EXIT***
